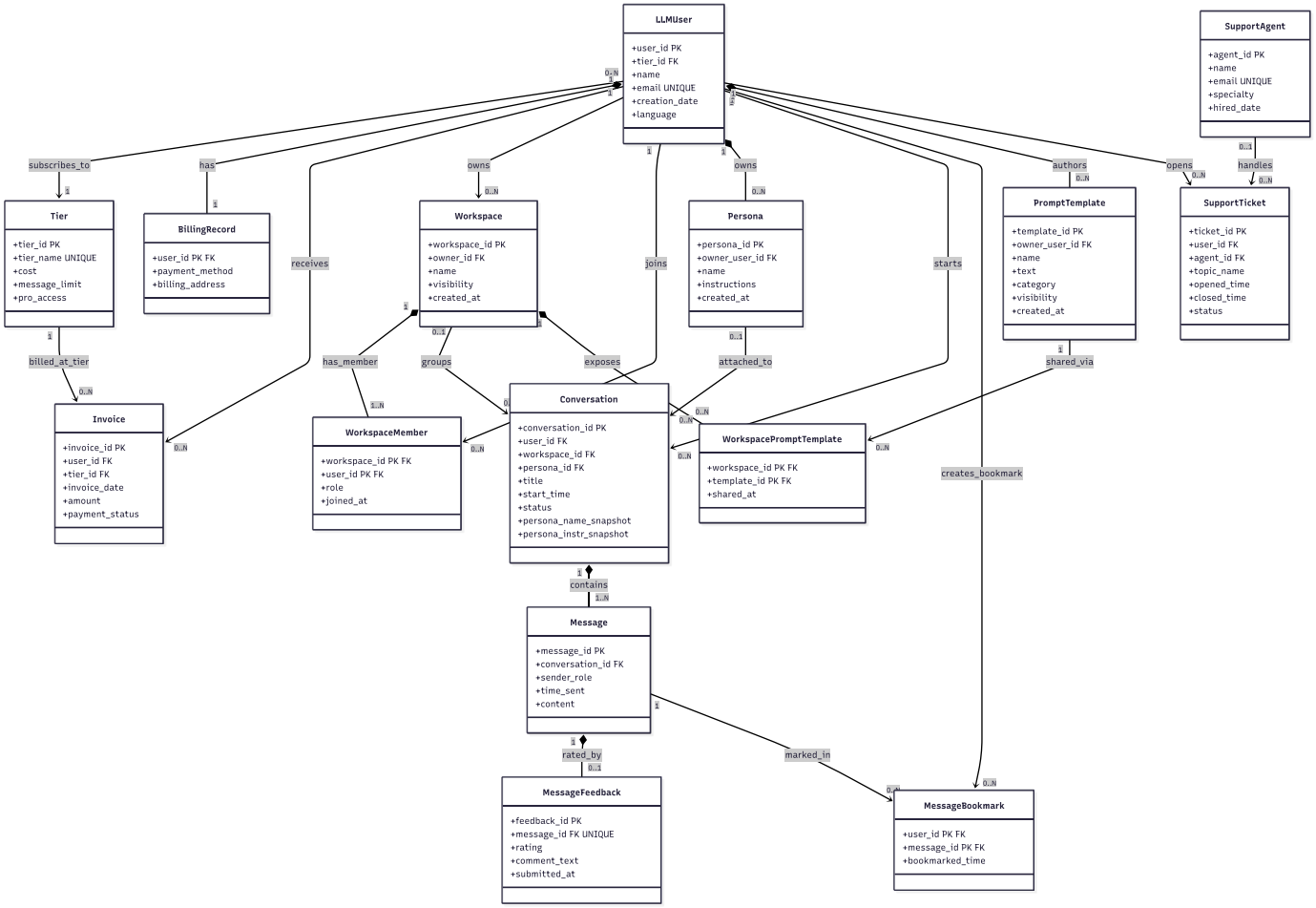
Design Document - Program #4

1. Conceptual Database Design

ER Diagram:



1.1 Entity Sets and Attributes

Entity	Attributes
Tier	tier_id (PK), tier_name (UNIQUE), cost, message_limit, pro_access
LLMUser	user_id (PK), tier_id (FK), name, email (UNIQUE), creation_date, language
BillingRecord	user_id (PK, FK), payment_method, billing_address
Invoice	invoice_id (PK), user_id (FK), tier_id (FK), invoice_date, amount, payment_status
Workspace	workspace_id (PK), owner_id (FK --> LLMUser), name, visibility, created_at
WorkspaceMember	workspace_id (FK), user_id (FK), role, joined_at - composite PK
Persona	persona_id (PK), owner_user_id (FK), name, instructions, created_at
Conversation	conversation_id (PK), user_id (FK), workspace_id (FK, nullable), persona_id (FK, nullable), title, start_time, status, persona_name_snapshot, persona_instr_snapshot
Message	message_id (PK), conversation_id (FK), sender_role, time_sent, content
MessageFeedback	feedback_id (PK), message_id (FK, UNIQUE), rating, comment_text, submitted_at
MessageBookmark	user_id (FK), message_id (FK), bookmarked_time - composite PK

Entity	Attributes
PromptTemplate	template_id (PK), owner_user_id (FK), name, text, category, visibility, created_at
WorkspacePromptTemplate	workspace_id (FK), template_id (FK), shared_at - composite PK
SupportAgent	agent_id (PK), name, email (UNIQUE), specialty, hired_date
SupportTicket	ticket_id (PK), user_id (FK), agent_id (FK, nullable), topic_name, opened_time, closed_time (nullable), status

1.2 Relationships and Cardinalities

Relationship	Entities	Cardinality	Notes
subscribes_to	LLMUser --> Tier	N:1, total	Every user has exactly one current tier.
billed_at_tier	Invoice --> Tier	N:1, total	Captures which tier was billed at invoice time -- decouples historical invoices from the user's current tier after upgrade/downgrade.
has	LLMUser <--> BillingRecord	1:1, total	Every user has exactly one billing record.
receives	LLMUser --> Invoice	1:N	
owns_workspace	LLMUser --> Workspace	1:N	Foreign key `owner_id`.
joins	LLMUser <--> Workspace	M:N (WorkspaceMember)	Attributes: `role`, `joined_at`.
groups	Workspace --> Conversation	1:N, optional	A conversation may exist outside any workspace.
owns (persona)	LLMUser --> Persona	1:N	
attached_to	Persona --> Conversation	1:N, optional	ON DELETE SET NULL; snapshot fields preserve history.
starts	LLMUser --> Conversation	1:N	
contains	Conversation --> Message	1:N, total on Message	
rated_by	Message --> MessageFeedback	1:0..1	Only AI-role messages may have feedback (enforced in application logic, ConversationMenu.java).
creates (bookmark)	LLMUser <--> Message	M:N (MessageBookmark)	Attribute: `bookmarked_time`.
authors	LLMUser --> PromptTemplate	1:N	
shared_via	PromptTemplate <--> Workspace	M:N (WorkspacePromptTemplate)	Templates with `visibility = 'SHARED'` may be exposed in one or more workspaces.
opens	LLMUser --> SupportTicket	1:N	
handles	SupportAgent --> SupportTicket	1:N, optional	`agent_id` nullable until assignment.

1.3 Design Rationale

1. **`Tier` as its own entity rather than an enum on User:** Tiers carry their own attributes (`message_limit`, `pro_access`, `cost`) that change independently of any specific user. Modeling it separately avoids update anomalies. When the Plus tier's daily limit changes, one row updates instead of every Plus user's record.

2. **`Invoice` carries a `tier_id` of its own:** An invoice represents a bill for a specific tier at a specific point in time. If the user's tier changes after billing, old invoices must not retroactively appear as if they were billed at the new tier. Giving Invoice its own `tier_id` FK decouples billing history from the user's current tier. The "Generate invoice" operation reads the user's current tier once, snapshots it onto the invoice, and subsequent tier changes leave the invoice untouched.

3. **`BillingRecord.user_id` is the primary key:** The spec mandates each user must have exactly one billing record; this is a strict 1:1 relationship with total participation. Using `user_id` as the PK enforces the 1:1 at the key level and eliminates a redundant surrogate key.

4. **`Persona` uses snapshots on Conversation:** The spec requires that if a Persona is updated or deleted, the historical context of existing conversations remains coherent. Three options were considered: (a) versioned Persona table (adds complexity and a table); (b) block updates and deletions (violates the deletion use case); (c) snapshot on attach (chosen).

   When a conversation is created with a persona, `persona_name_snapshot` and `persona_instr_snapshot` are copied from the persona into the conversation row. The FK `persona_id` is retained for analytics (Query 3) with ON DELETE SET NULL. The snapshot fields are not functionally determined by `persona_id` -- they record state at attach time, not current state -- making this legitimate point-in-time data analogous to `price_at_purchase` in an order table, not a 3NF violation.

5. **`Conversation.status` is useful:** The spec defines persona deletion in terms of "ongoing conversations." Without a status attribute, every conversation is treated as ongoing forever. `status ∈ {'ACTIVE', 'ARCHIVED'}` gives the persona-deletion guard (`COUNT WHERE persona_id = X AND status = 'ACTIVE' > 5`) a meaningful, manageable interpretation and provides a natural UX for archiving old threads.

6. **`MessageFeedback` is a separate relation, not columns on Message:** Feedback is optional and only applies to AI messages. Nullable `rating` and `comment_text` columns on every Message would waste space and muddle semantics. As a separate `1:0..1` table, the presence of a row itself encodes "feedback was given." `MessageFeedback` uses a surrogate `feedback_id` with `UNIQUE(message_id)` rather than `message_id` as PK, to continue enforcing the `1:0..1` rule.

7. **`PromptTemplate` uses an associative `WorkspacePromptTemplate`:** A prompt template can be shared across multiple workspaces without copying it -- one template row, many workspaces linked to it. The visibility field ('PRIVATE' or 'SHARED') acts as a quick label so you don't have to check the junction table every time. Application logic keeps them consistent: if a template is marked PRIVATE, it can't be added to any workspace (enforced in PromptMenu.java).

8. **`SupportTicket` has no `resolution_duration` column:** Storing `resolution_duration` alongside `opened_time` and `closed_time` would create the FD `{opened_time, closed_time} --> resolution_duration` where the LHS is not a superkey. This is a 3NF violation. Duration is computed at query time (`closed_time - opened_time`). The terminal outcome (Resolved/Escalated) is folded into the `status` column to avoid storing the same information in two places.

1.4 Additional Constraints not in E-R Diagram

These business rules are enforced in application logic (JDBC):

1. User deletion must fail if the user has any `Unpaid` invoice or any ticket with `status IN ('Open', 'In Progress')`. Enforced by pre-delete checks in UserMenu.java.
2. Persona deletion must fail if more than five conversations reference it with `status = 'ACTIVE'`. Enforced by a COUNT guard in `PersonaMenu.java`.
3. Rate-limit check on message insert: before inserting a USER-role message, the application counts today's USER-role messages for that user across all conversations and compares against `Tier.message_limit`. Enforced in `ConversationMenu.java`.
4. Workspace membership check before moving a conversation: verify a row exists in `WorkspaceMember` for `(target_workspace_id, conversation_owner_id)`. Enforced in `WorkspaceMenu.java`.
5. Feedback role restriction: `MessageFeedback` rows may only exist for messages where `Message.sender_role = 'AI'`. Enforced by a pre-insert check in `ConversationMenu.java` (`updateFeedback`).
6. PromptTemplate visibility consistency: a PRIVATE template cannot be added to `WorkspacePromptTemplate`. Enforced by a pre-insert check in `PromptMenu.java` (`shareTemplate`).
7. SupportTicket status/closed_time consistency: `status IN ('Resolved','Escalated')` if and only if `closed_time IS NOT NULL`. Enforced by table-level CHECK constraint `chk_ticket_closed` in `create_tables.sql`.

8. Cascade behavior:
    1. `ON DELETE CASCADE` from `LLMUser` --> `Conversation` --> `Message` --> `MessageFeedback` / `MessageBookmark`
    2. `ON DELETE SET NULL` from `Workspace` --> `Conversation.workspace_id` and `Persona` --> `Conversation.persona_id`;
    3. **no** `ON DELETE` clause (defaults to `RESTRICT`) on `Tier` --> `LLMUser` and `Tier` --> `Invoice`.

2. Logical Database Design

2.1 ER-to-Relational Conversion

1. Every regular entity becomes one table whose PK is the ER identifier.
2. The 1:1 relationship LLMUser <--> BillingRecord is merged into BillingRecord with `user_id` as PK+FK (no separate junction).
3. 1:N relationships become FKs on the N-side: `user_id` on Conversation, Invoice, Persona, PromptTemplate, SupportTicket; `workspace_id` on Conversation; `persona_id` on Conversation; `agent_id` on SupportTicket; `tier_id` on LLMUser and Invoice.
4. M:N relationships become associative tables with composite PKs: `WorkspaceMember`, `MessageBookmark`, `WorkspacePromptTemplate`.
5. Optional participation becomes nullable FKs: Conversation.workspace_id, Conversation.persona_id, SupportTicket.agent_id, SupportTicket.closed_time.
6. The 1:0..1 Message <--> MessageFeedback is realized as a separate relation keyed by surrogate `feedback_id` with `UNIQUE(message_id)`.
7. Derived attributes like resolution duration are not stored; they are computed in queries.

2.2 Relational Schema

All attributes NOT NULL unless noted as nullable.

Relation	Primary Key	Foreign Keys	Other Attributes	Notes
Tier	tier_id	-	tier_name (UNIQUE), cost, message_limit, pro_access (CHECK IN ('Y','N'))	-
LLMUser	user_id	tier_id --> Tier	name, email (UNIQUE), creation_date, language	-
BillingRecord	user_id	user_id --> LLMUser (CASCADE)	payment_method, billing_address	PK is also the FK
Invoice	invoice_id	user_id --> LLMUser (CASCADE), tier_id --> Tier	invoice_date, amount, payment_status (CHECK IN ('Paid','Unpaid','Overdue'))	-
Workspace	workspace_id	owner_id --> LLMUser (CASCADE)	name, visibility (CHECK IN ('PRIVATE','SHARED')), created_at	-
WorkspaceMember	(workspace_id, user_id)	workspace_id --> Workspace (CASCADE), user_id --> LLMUser (CASCADE)	role (CHECK IN ('OWNER','MEMBER')), joined_at	Composite PK
Persona	persona_id	owner_user_id --> LLMUser (CASCADE)	name, instructions, created_at	UNIQUE (owner_user_id, name)
Conversation	conversation_id	user_id --> LLMUser	title, start_time, status (CHECK IN ('ACTIVE','ARCHIVED')),	-

Relation	Primary Key	Foreign Keys	Other Attributes	Notes
		(CASCADE), workspace_id --> Workspace (SET NULL, nullable), persona_id --> Persona (SET NULL, nullable)	persona_name_snapshot (nullable), persona_instr_snapshot (nullable)	
Message	message_id	conversation_id --> Conversation (CASCADE)	sender_role (CHECK IN ('USER','AI')), time_sent, content	-
MessageFeedback	feedback_id	message_id --> Message (CASCADE, UNIQUE)	rating (CHECK IN ('Thumbs Up','Thumbs Down')), comment_text (nullable), submitted_at	App check (ConversationMenu.java): message must have sender_role = 'AI'
MessageBookmark	(user_id, message_id)	user_id --> LLMUser (CASCADE), message_id --> Message (CASCADE)	bookmarked_time	Composite PK
PromptTemplate	template_id	owner_user_id --> LLMUser (CASCADE)	name, text, category, visibility (CHECK IN ('PRIVATE','SHARED')), created_at	-
WorkspacePromptTemplate	(workspace_id, template_id)	workspace_id --> Workspace (CASCADE), template_id --> PromptTemplate (CASCADE)	shared_at	Composite PK; app check (PromptMenu.java): template must have visibility = 'SHARED'
SupportAgent	agent_id	-	name, email (UNIQUE), specialty, hired_date	-
SupportTicket	ticket_id	user_id --> LLMUser (RESTRICT), agent_id --> SupportAgent (SET NULL, nullable)	topic_name (CHECK IN ('Billing','Model Error','Account','Feature Request','Other')), opened_time, closed_time (nullable), status (CHECK IN ('Open','In Progress','Resolved','Escalated'))	CHECK: status IN ('Resolved','Escalated') iff closed_time IS NOT NULL

3. Normalization Analysis

Every relation is in BCNF, which implies 3NF. What follows is an analysis of the FDs of each relation.

Tier
FDs:

1. `tier_id --> tier_name, cost, message_limit, pro_access`
2. `tier_name --> tier_id, cost, message_limit, pro_access`

Candidate keys: `{tier_id}, {tier_name}`.
Both are superkeys. Cost, message_limit, and pro_access are independent facts about the tier, and none determines another. Thus, both BCNF and 3NF

LLMUser
FDs:

1. `user_id --> tier_id, name, email, creation_date, language`
2. `email --> user_id, tier_id, name, creation_date, language`

Candidate keys: `{user_id}, {email}`.
Both are superkeys. `tier_id` does not determine any other User attribute -- two users can share a tier and differ in every other column. Thus, both BCNF and 3NF.

BillingRecord
FDs:

1. `user_id --> payment_method, billing_address`

Candidate keys: `{user_id}`.
Single FD with PK as LHS. Thus, both BCNF and 3NF.

Invoice
FDs:

1. `invoice_id --> user_id, tier_id, invoice_date, amount, payment_status`

Candidate keys: `{invoice_id}`.
`tier_id` does NOT determine `amount` -- amount is historical and may reflect prorated or discounted values. `invoice_date` does not determine `payment_status`. No non-key FDs exist. Thus, both BCNF and 3NF.

Workspace
FDs:

1. `workspace_id --> owner_id, name, visibility, created_at`

Candidate keys: `{workspace_id}`.
`name` is not unique (different owners may have same-name workspaces). Thus, both BCNF and 3NF.

WorkspaceMember
FDs:

1. `{workspace_id, user_id} --> role, joined_at`

Candidate keys: `{workspace_id, user_id}`.
Only non-trivial FD has the composite PK as LHS. Thus, both BCNF and 3NF.

Persona
FDs:

1. `persona_id --> owner_user_id, name, instructions, created_at`
2. `{owner_user_id, name} --> persona_id, instructions, created_at`

Candidate keys: `{persona_id}, {owner_user_id, name}` (enforced by `UNIQUE` constraint).
Both are superkeys. Thus, both BCNF and 3NF.

Conversation
FDs:

1. `conversation_id --> user_id, workspace_id, persona_id, title, start_time, status, persona_name_snapshot, persona_instr_snapshot`

Candidate keys: `{conversation_id}`.

Note: `persona_id --> persona_name_snapshot` is NOT a valid FD in this relation. The snapshot columns record the persona's state at the moment of attachment. Because personas can be updated after attachment, two conversations with the same

`persona_id` attached at different times may carry different snapshot values.

Thus, both BCNF and 3NF.

Message
FDs:

1. `message_id --> conversation_id, sender_role, time_sent, content`

Candidate keys: `{message_id}`.
Single FD with PK as LHS. Thus, both BCNF and 3NF.

MessageFeedback
FDs:

1. `feedback_id --> message_id, rating, comment_text, submitted_at`
2. `message_id --> feedback_id, rating, comment_text, submitted_at`

Candidate keys: `{feedback_id}, {message_id}` (`UNIQUE` constraint on `message_id` makes it a second candidate key).
Both LHSes are superkeys. Thus, both BCNF and 3NF.

MessageBookmark
FDs:

1. `{user_id, message_id} --> bookmarked_time`

Candidate keys: `{user_id, message_id}`.
Single FD with composite PK as LHS. Thus, both BCNF and 3NF.

PromptTemplate
FDs:

1. `template_id --> owner_user_id, name, text, category, visibility, created_at`

Candidate keys: `{template_id}`.
`category` is a free-form user label, not a determinant. `visibility` is a simple flag independent of other attributes. Thus, both BCNF and 3NF.

WorkspacePromptTemplate
FDs:

1. `{workspace_id, template_id} --> shared_at`

Candidate keys: `{workspace_id, template_id}`.
Single FD with composite PK as LHS. Thus, both BCNF and 3NF.

SupportAgent
FDs:

1. `agent_id --> name, email, specialty, hired_date`
2. `email --> agent_id, name, specialty, hired_date`

Candidate keys: `{agent_id}, {email}`.
Both are superkeys. Thus, both BCNF and 3NF.

SupportTicket
FDs:

1. `ticket_id --> user_id, agent_id, topic_name, opened_time, closed_time, status`

Candidate keys: `{ticket_id}`.
Note: we discuss earlier why we do not store a `resolution_duration` column.

Both BCNF and 3NF.

4. Self-Designed Query Description

Question:

> For a given user (provided by the operator), list all conversations that were started with a persona attached, along with the persona name used, the conversation title, the start time, and the number of AI messages in that conversation.

SQL:

```sql
SELECT c.title,
       c.start_time,
       c.persona_name_snapshot,
       COUNT(m.message_id) AS ai_message_count
FROM   Conversation c
JOIN   Message m
    ON m.conversation_id = c.conversation_id
   AND m.sender_role = 'AI'
WHERE  c.user_id = ?
  AND  c.persona_id IS NOT NULL
GROUP  BY c.title, c.start_time, c.persona_name_snapshot
ORDER  BY c.start_time DESC
```

The bind parameter `?` is filled from user input (the operator enters a User ID at the prompt).

Requirement Satisfaction
It statisfies the requirements becaseuse:

1. **It has more than two relations:** three relations are joined: `Conversation`, `Message`, and implicitly `LLMUser` (the user whose ID is supplied filters Conversation rows that reference LLMUser.user_id via the FK).
2. **It uses user-provided information:** the User ID is entered interactively by the operator at runtime and bound as a parameter

Utility of this Query
This query helps an operator or user understand their personalization history: which personas they have used, how heavily each conversation engaged the AI (AI message count), and in what order they were started. Some examples of uses:

1. **User self-service:** a user reviewing their account can see which persona-linked conversations are most active and decide whether to archive old ones.
2. **Support context:** a support agent handling a billing or usage question can quickly see which personas are associated with a user's recent conversations.
3. **Persona adoption audit:** product managers can see, per user, whether persona-linked conversations generate more AI responses than non-persona conversations (by comparing this query's output to a parallel query without the `persona_id IS NOT NULL` filter).